

# Lab 9

## LabSection.java

```
LabSection.java : RegisterRunnable.j... : UnregisterRunnabl... : progDemo.java :
1 public class LabSection {
2
3     private int numOfReg;
4
5     public LabSection(int r) {
6         numOfReg = r;
7     }
8
9     public synchronized void register() {
10         int updNoOfReg = numOfReg + 1;
11
12         System.out.println("Register-request, registered students: "
13             + updNoOfReg);
14
15         numOfReg = updNoOfReg;
16     }
17
18     public synchronized void unregister() {
19         int updNoOfReg = numOfReg - 1;
20
21         System.out.println("Unregister-request, registered students: "
22             + updNoOfReg);
23
24         numOfReg = updNoOfReg;
25     }
26
27     public int numOfRegisteredStudents() {
28         return numOfReg;
29     }
30 }
```

## RegisterRunnable.java

```
LabSection.java : RegisterRunnable.j... : UnregisterRunnabl... : progDemo.java :
1 public class RegisterRunnable implements Runnable {
2
3     private static final int DELAY = 200;
4     private LabSection osLabGroup;
5
6     public RegisterRunnable(LabSection osLab) {
7         osLabGroup = osLab;
8     }
9
10    public void run() {
11        try {
12            osLabGroup.register();
13            Thread.sleep(DELAY);
14        } catch (InterruptedException exception) {
15        }
16    }
17 }
```

# Lab 9

## UnregisterRunnable.java

```
LabSection.java : RegisterRunnable.j... : UnregisterRunnabl... : progDemo.java :
1 public class UnregisterRunnable implements Runnable {
2
3     private static final int DELAY = 200;
4     private LabSection osLabGroup;
5
6     public UnregisterRunnable(LabSection osLab) {
7         osLabGroup = osLab;
8     }
9
10    public void run() {
11        try {
12            osLabGroup.unregister();
13            Thread.sleep(DELAY);
14        } catch (InterruptedException exception) {
15        }
16    }
17 }
```

## progDemo.java

```
LabSection.java : RegisterRunnable.j... : UnregisterRunnabl... : progDemo.java :
1 public class progDemo {
2
3     public static void main(String[] args) {
4
5         LabSection osLab = new LabSection(25);
6
7         for (int i = 1; i <= 25; i++) {
8
9             RegisterRunnable reRun = new RegisterRunnable(osLab);
10            UnregisterRunnable unRegRun = new UnregisterRunnable(osLab);
11
12            Thread regThrd = new Thread(reRun);
13            Thread unRegThrd = new Thread(unRegRun);
14
15            regThrd.start();
16            unRegThrd.start();
17        }
18
19        try {
20            Thread.sleep(3000);
21        } catch (InterruptedException exception) {
22        }
23
24        System.out.println("Final number of registered students: "
25                           + osLab.numOfRegisteredStudents());
26    }
27 }
```

# Lab 9

## The OutPut:

[illegible]

```
...Program finished with exit code 0
Press ENTER to exit console.
```

# Lab 9

## BankAccount.java

```
BankAccount.java : DepositRunnable.j... : WithdrawRunnable... : BankAccountThrea... :
1 public class BankAccount {
2
3     private double balance;
4
5     public BankAccount() {
6         balance = 0;
7     }
8
9     public synchronized void deposit(double amount) {
10         double newBalance = balance + amount;
11
12         System.out.println("Depositing " + amount +
13             ", new balance is: " + newBalance);
14
15         balance = newBalance;
16     }
17
18     public synchronized void withdraw(double amount) {
19         double newBalance = balance - amount;
20
21         System.out.println("Withdrawing " + amount +
22             ", new balance is: " + newBalance);
23
24         balance = newBalance;
25     }
26
27     public double getBalance() {
28         return balance;
29     }
30 }
```

## DepositRunnable.java

```
BankAccount.java : DepositRunnable.j... : WithdrawRunnable... : BankAccountThrea... :
1 public class DepositRunnable implements Runnable {
2
3     private static final int DELAY = 200;
4     private BankAccount account;
5     private double amount;
6
7     public DepositRunnable(BankAccount anAccount, double anAmount) {
8         account = anAccount;
9         amount = anAmount;
10    }
11
12    public void run() {
13        try {
14            account.deposit(amount);
15            Thread.sleep(DELAY);
16        } catch (InterruptedException exception) {
17        }
18    }
19 }
```



# Lab 9

## WithdrawRunnable.java

```
BankAccount.java : DepositRunnable.j... : WithdrawRunnable... : BankAccountThrea... :
1- public class WithdrawRunnable implements Runnable {
2-
3-     private static final int DELAY = 200;
4-     private BankAccount account;
5-     private double amount;
6-
7-     public WithdrawRunnable(BankAccount anAccount, double anAmount) {
8-         account = anAccount;
9-         amount = anAmount;
10-    }
11-
12-    public void run() {
13-        try {
14-            account.withdraw(amount);
15-            Thread.sleep(DELAY);
16-        } catch (InterruptedException exception) {
17-        }
18-    }
19- }
```

## BankAccountThreadRunner.java

```
BankAccount.java : DepositRunnable.j... : WithdrawRunnable... : BankAccountThrea... :
1- public class BankAccountThreadRunner {
2-
3-     public static void main(String[] args) {
4-
5-         BankAccount account = new BankAccount();
6-
7-         final double AMOUNT = 100;
8-         final int THREADS = 25;
9-
10-        for (int i = 1; i <= THREADS; i++) {
11-
12-            DepositRunnable d = new DepositRunnable(account, AMOUNT);
13-            WithdrawRunnable w = new WithdrawRunnable(account, AMOUNT);
14-
15-            Thread dt = new Thread(d);
16-            Thread wt = new Thread(w);
17-
18-            dt.start();
19-            wt.start();
20-        }
21-
22-        try {
23-            Thread.sleep(3000);
24-        } catch (InterruptedException exception) {
25-        }
26-
27-        System.out.println("Final Balance: " + account.getBalance());
28-    }
29- }
```

# Lab 9

## The OutPut:

```

Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Withdrawing 100.0, new balance is: -100.0
Depositing 100.0, new balance is: 0.0
Withdrawing 100.0, new balance is: -100.0
Depositing 100.0, new balance is: 0.0
Withdrawing 100.0, new balance is: -100.0
Depositing 100.0, new balance is: 0.0
Withdrawing 100.0, new balance is: -100.0
Depositing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Withdrawing 100.0, new balance is: -100.0
Withdrawing 100.0, new balance is: -200.0
Depositing 100.0, new balance is: -100.0
Depositing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Withdrawing 100.0, new balance is: -100.0
Depositing 100.0, new balance is: 0.0
Withdrawing 100.0, new balance is: -100.0
Withdrawing 100.0, new balance is: -200.0
Depositing 100.0, new balance is: -100.0
Withdrawing 100.0, new balance is: -200.0
Depositing 100.0, new balance is: -100.0
Depositing 100.0, new balance is: 0.0
Withdrawing 100.0, new balance is: -100.0
Depositing 100.0, new balance is: 0.0
Withdrawing 100.0, new balance is: -100.0
Depositing 100.0, new balance is: 0.0
Depositing 100.0, new balance is: 100.0
Withdrawing 100.0, new balance is: 0.0
Withdrawing 100.0, new balance is: -100.0
Depositing 100.0, new balance is: 0.0
Withdrawing 100.0, new balance is: -100.0
Depositing 100.0, new balance is: 0.0
Final Balance: 0.0

```

```
...Program finished with exit code 0
```